Auto-Scaling E-Commerce Backend with Chaos Engineering

A Resilient Microservices Platform on AWS with Managed Fault Injection

Course	Cloud Computing (CE-408) — Semester Project
Instructor	Ms. Safia Baloch
Institution	Ghulam Ishaq Khan Institute (GIKI)
Submission	Project Proposal (Revised)
Team Members	Ahmed Musharaf (2022067), Abdullah Saeed (2022328), Abdullah Zafar (2022329)
Target Platform	Amazon Web Services (AWS)

1. Project Overview

We propose to design and deploy a resilient e-commerce backend composed of three microservices — **Catalog** (product browsing with inventory checks), **Cart** (per-user shopping sessions), and **Orders** (checkout and asynchronous fulfilment). Each service runs as an independently scalable container on AWS ECS Fargate behind an Application Load Balancer, with data stores chosen per service workload (PostgreSQL, DynamoDB). The platform auto-scales horizontally based on CPU utilization, and a synthetic load generator (k6) simulates baseline shopping traffic and sudden spike loads.

The differentiator that elevates this beyond an assignment is the **chaos engineering harness**: using AWS Fault Injection Service, we deliberately terminate Fargate tasks and inject network latency while traffic is live. CloudWatch dashboards visualize how the system degrades and recovers in real time, and a Service Level Objective (SLO) report quantifies the resilience of each scenario. The project demonstrates not just that microservices can be deployed in the cloud, but that they survive the failures cloud environments routinely produce.

2. System Architecture (AWS)

Customer-facing API requests terminate at an Application Load Balancer and are path-routed to the appropriate ECS Fargate service. The Catalog service reads from RDS PostgreSQL. The Cart service uses DynamoDB for low-latency session storage keyed by user ID. The Orders service writes to RDS PostgreSQL and emits an SQS message for asynchronous fulfilment processing. Each service has its own auto-scaling policy targeting 60% CPU. AWS FIS templates run on a schedule during demonstration windows, while k6 generates baseline and spike traffic. CloudWatch correlates metrics and logs so that the impact of each chaos experiment is visible in a single dashboard.

Layer	AWS Services
API & Compute	Application Load Balancer, ECS Fargate (Catalog, Cart, Orders services), Auto Scaling (target-tracking on CPU), ECR (image registry)
Data & Messaging	RDS PostgreSQL (Catalog, Orders), DynamoDB (Cart sessions), SQS (order processing queue)

Resilience & Chaos	AWS Fault Injection Service (FIS) for managed chaos experiments, k6 (load generation from EC2), CloudWatch dashboards & alarms
Foundations	VPC, IAM, Security Groups, CloudWatch Logs

3. Work Breakdown

The three workstreams are intentionally decoupled through stable contracts: REST API specifications between services, infrastructure-as-code task definitions for handoff, and a documented SLO baseline that the chaos workstream measures against. Each member can develop and validate their portion independently, with integration milestones at the end of week one.

Member	Role	Primary Responsibilities
Ahmed Musharaf (2022067)	Infrastructure, Microservices & Chaos	Overall architecture ownership and integration. ECS Fargate cluster, task definitions, ALB routing, auto-scaling policies, ECR pipeline, IAM and VPC setup. Catalog and Orders microservices (Python/FastAPI) including RDS schema and SQS-based order processing. Authors AWS FIS experiment templates (task termination, latency injection) and integrates them with the live system.
Abdullah Saeed (2022328)	Data Layer & API Contracts	Cart microservice with DynamoDB session store, REST API contract definition and enforcement across all three services, integration testing harness, and shared client utilities used by k6.
Abdullah Zafar (2022329)	Load Testing & Observability	k6 load test scenarios (baseline traffic and spike loads), CloudWatch dashboards, SLO definitions, and the resilience report capturing recovery times and error-rate impact for each chaos experiment.

4. Deliverables and Evaluation

- **Deployable platform**: the full backend running on AWS, reproducible from a documented setup script, with all three microservices accessible through a single load-balanced endpoint.
- **Chaos experiment catalogue**: a documented set of AWS FIS experiment templates covering task termination and network latency injection, each with a hypothesis, steady-state metric, and rollback plan.
- **Live demonstration**: real-time chaos experiments executed against the running platform under k6 load, with the CloudWatch dashboard projected to show degradation and recovery as it happens.

- **Source repositories**: microservices code, infrastructure configuration, FIS templates, and load-test scripts, all version-controlled with brief READMEs.
- **Resilience report**: a written analysis covering the architecture, the auto-scaling behaviour observed under k6 spike tests, the recovery time and error-rate impact of each chaos experiment, and the resulting SLO compliance.

5. Project Timeline

Phase 1 — Build: Days 1–2 cover AWS account preparation, VPC and ECR setup, and scaffolding the three microservices with mock data. Days 3–4 wire up the data stores, ALB routing, and ECS task definitions; an end-to-end happy path is demonstrated by day 4. Days 5–7 add auto-scaling policies and a working k6 baseline test.

Phase 2 — Break and Measure: Days 8–10 author and run the AWS FIS experiments, capturing CloudWatch traces for each scenario. Days 11–12 build the dashboard, refine SLOs, and tune auto-scaling based on observed behaviour. Days 13–14 are reserved for the resilience report, demo rehearsal, and a full end-to-end dry run.

6. Future Work (Stretch Goals)

The following components were considered during scoping but moved out of the committed plan to keep the core resilience-engineering thesis deliverable within the project window. They will be implemented if time permits after the core scope is complete and verified.

- **Edge layer (CloudFront + Route 53)**: a CDN tier in front of the ALB with a custom domain. Deferred because the chaos demonstration is fully observable through the ALB DNS and adds CloudFront propagation overhead to every infrastructure iteration.
- **Cognito authentication**: a managed user pool issuing JWTs for the Cart and Orders services. Deferred in favour of a static API key during development; the architecture leaves a clear seam for adding Cognito later.
- **ElastiCache Redis caching layer**: a hot-product cache in front of the Catalog service's RDS reads. Deferred because Catalog read performance is acceptable under the planned load profile, and the cache adds a non-trivial failure mode to reason about during chaos experiments.
- **X-Ray distributed tracing**: end-to-end request traces correlated across all three services. Deferred because CloudWatch metrics and logs are sufficient to quantify the SLO impact of each chaos experiment; X-Ray would deepen the analysis but is not required for it.
- **Additional chaos experiments**: CPU stress on Fargate tasks and full Availability Zone failure simulation. The committed scope covers task termination and latency injection, which together exercise the auto-scaling, ALB target health, and inter-service timeout behaviour that the resilience report is built around. The additional experiments would extend the catalogue but not change its conclusions.
- **React storefront**: a minimal browser-based UI for the demo. Deferred because the live demonstration is driven by k6 traffic and the CloudWatch dashboard, which more directly shows the chaos behaviour than a UI would.
- **Request-rate-based auto-scaling**: a second target-tracking policy on ALB request count per target. Deferred in favour of CPU-only scaling, which is simpler to reason about during chaos experiments; request-rate scaling would be added once baseline behaviour is well understood.